Assignment-13.3

2303A51304

Batch-05

Task-01:

Prompt:

Refactor the following legacy Python script to remove code duplication and improve maintainability.

use coding practices use functions and required parameters.

The program should still print the area and perimeter for all given rectangles.

Legacy Code:

```
print("Area of Rectangle:", 5 * 10)

print("Perimeter of Rectangle:", 2 * (5 + 10))

print("Area of Rectangle:", 7 * 12)

print("Perimeter of Rectangle:", 2 * (7 + 12))

print("Area of Rectangle:", 10 * 15)

print("Perimeter of Rectangle:", 2 * (10 + 15))
```

Output:Provide the fully refactored Python code only.

Code:

```
> task-01.py > ...
'''Refactor the following legacy Python script to remove code duplication and improve maintaina
use coding practices use functions and required parameters.
The program should still print the area and perimeter for all given rectangles.

Legacy Code:
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
Output:Provide the fully refactored Python code only.'''''
def calculate_area(length, width):
    return length * width

def calculate_perimeter(length, width):
    return 2 * (length + width)

# Example usage:
print("Area of Rectangle:", calculate_area(5, 10))
print("Perimeter of Rectangle:", calculate_perimeter(5, 10))
print("Area of Rectangle:", calculate_area(7, 12))
print("Perimeter of Rectangle:", calculate_perimeter(7, 12))
print("Area of Rectangle:", calculate_area(10, 15))
print("Perimeter of Rectangle:", calculate_perimeter(10, 15))
```

Output:

```
Area of Rectangle: 50
PS C:\Users\pooji\OneDrive\Desktop\AI_Assistance_coding> & C:/Users/pooji/AppData
rs/pooji/OneDrive/Desktop/AI_Assistance_coding/lab-13/task-01.py
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
PS C:\Users\pooji\OneDrive\Desktop\AI_Assistance_coding>
```

Explanation:

This program removes repeated calculations by creating reusable functions instead of writing the same formulas multiple times.

The calculate_area() function computes the rectangle's area using length and width as inputs.

The calculate_perimeter() function calculates the perimeter using the standard formula.

Each rectangle's dimensions are passed as parameters, making the code cleaner and easier to reuse. This approach improves readability, reduces duplication, and makes future changes simple to implement.

Task-02:

Prompt:

Refactor the following Python script to optimize performance by removing inefficient nested loops.

Replace nested loops with an optimized approach using sets or list comprehensions and also

Improve time complexity and explain why the new solution is faster and the program output exactly the same.

Include a short performance comparison between the original and refactored versions (time complexity explanation).

Legacy Code:

```
names = ["Alice", "Bob", "Charlie", "David"]

search_names = ["Charlie", "Eve", "Bob"]

for s in search_names:

    found = False
```

for n in names:

if s == n:

found = True

if found:

print(f"{s} is in the list")

else:

print(f"{s} is not in the list")

Output: Provide the optimized Python code followed by a brief performance comparison

Code:

```
# Optimized Version (Refactored Code)

# Original data
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]

# Convert list to set for faster lookup (O(1) average time)
names_set = set(names)

# Check presence of each name
for s in search_names:
    if s in names_set:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
```

```
Charlie is in the list
Eve is not in the list
Bob is in the list
```

Output:

```
•••  Charlie is in the list
     Eve is not in the list
     Bob is in the list
```

Explanation:

Original Code Time Complexity: O(n*m) where n is the length of search_names and m is the length of names due to nested loops.

Refactored Code Time Complexity: O(n) for converting the list to a set and

O(1) for each membership check, resulting in O(n) overall for the search_names list.

The new solution is faster because it eliminates the need for nested loops, allowing for constant time membership checks after an initial linear time conversion to a set.

Task-03:

Prompt:

Refactor the following legacy Python script by extracting repeated calculations into reusable functions.

Create a function named calculate_total(price).

The function should calculate tax (18%) and return the total price including tax.

The program output must remain the same.

Legacy Code:

price = 250

tax = price * 0.18

total = price + tax

print("Total Price:", total)

price = 500

tax = price * 0.18

total = price + tax

print("Total Price:", total)

Output:Provide the fully refactored Python code only.

Code:

```
'''Refactor the following legacy Python script by extracting repeated calculations in

Create a function named calculate_total(price).
The function should calculate tax (18%) and return the total price including tax.
The program output must remain the same.

Legacy Code:
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)

price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)

Output:Provide the fully refactored Python code only.'''''
def calculate_total(price):
    tax = price * 0.18
    total = price + tax
    return total
# Example usage:
print("Total Price:", calculate_total(250))
print("Total Price:", calculate_total(500))
```

Output:

```
PS C:\Users\pooji\OneDrive\Desktop\AI_Assistance_coding> & C:/Users/pooji/AppData/
rs/pooji/OneDrive/Desktop/AI_Assistance_coding/lAssign-12.4/task-03.py
Total Price: 295.0
Total Price: 590.0
PS C:\Users\pooji\OneDrive\Desktop\AI_Assistance_coding>
```

Explanation:

The refactored code defines a function calculate_total that takes the price as an argument and calculates the total price including tax.

This eliminates the need for repeated code for tax calculation and total price computation, improving maintainability and readability.

The time complexity of both the original and refactored versions is O(1) for each price calculation, as they perform a constant number of operations.

However, the refactored version is more efficient in terms of code reuse and reduces the chances of errors when making changes to the tax calculation logic in the future.

Task-04

Prompt:

Refactor the following Python script by replacing hardcoded values (magic numbers) with named constants.

Requirements:

Create constants such as PI and RADIUS using uppercase naming conventions.

Improve readability and maintainability of the code.

Follow clean code practices.

The program output must remain exactly the same.

Legacy Code:

print("Area of Circle:", 3.14159 * (7 ** 2))

print("Circumference of Circle:", 2 * 3.14159 * 7)

Output:

Provide the fully refactored Python code only.

Code:

```
'''Refactor the following Python script by replacing hardcoded values (magic numbers) with n

Requirements:
    Create constants such as PI and RADIUS using uppercase naming conventions.
      Improve readability and maintainability of the code.
      Follow clean code practices.
      The program output must remain exactly the same.

Legacy Code:
print("Area of Circle:", 3.14159 * (7 ** 2))
print("Circumference of Circle:", 2 * 3.14159 * 7)

Output:
Provide the fully refactored Python code only.'''
PI = 3.14159
RADIUS = 7
print("Area of Circle:", PI * (RADIUS ** 2))
print("Circumference of Circle:", 2 * PI * RADIUS)
```

Output:

```
PS C:\Users\pooji\OneDrive\Desktop\AI_Assistance_coding> & C:/User
rs/pooji/OneDrive/Desktop/AI_Assistance_coding/lab-13/task-04.py
Area of Circle: 153.93791
Circumference of Circle: 43.98226
PS C:\Users\pooji\OneDrive\Desktop\AI_Assistance_coding>
```

Explanation:

The refactored code defines constants PI and RADIUS, which replace the hardcoded values in the original code.

This improves readability by giving meaningful names to the values, making it clear what they represent.

It also enhances maintainability, as any future changes to the value of PI or RADIUS can be made in one place without needing to search through the code for all occurrences.

The time complexity of both the original and refactored versions is O(1) since they perform a constant number of operations, but the refactored version is more maintainable and easier to understand.

Task-05:

Prompt:

Refactor the following Python script to improve variable naming and overall readability.

Requirements:

Replace unclear variable names with descriptive and meaningful names.

Add inline comments explaining the logic of the calculation.

Do not change the programs functionality.The output must remain exactly the same.

Legacy Code:

a = 10

b = 20

c = a * b / 2

print(c)

Output:

Provide the fully refactored Python code only.

Code:

```
Do not change the programs functionality.The output must remain exactly the s
Legacy Code:
a = 10
b = 20
c = a * b / 2
print(c)
Output:
Provide the fully refactored Python code only.'''

# Define the base and height of the triangle
base = 10
height = 20
# Calculate the area of the triangle using the formula: (base * height) / 2
area_of_triangle = base * height / 2
# Print the calculated area
print(area_of_triangle)
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN

rs/pooji/OneDrive/Desktop/AI_Assistance_coding/lab-13/task-0
100.0
PS C:\Users\pooji\OneDrive\Desktop\AI_Assistance_coding>
```

Explanation:

The refactored code uses descriptive variable names (base, height, area_of_triangle) that clearly indicate their purpose, improving readability.

Inline comments are added to explain the logic of the calculation, making it easier for others to understand the code at a glance.

The time complexity of both the original and refactored versions is O(1) since they perform a constant number of operations.

However, the refactored version is more maintainable and easier to understand due to improved variable naming and added comments.